

Plataforma de Delivery Multiestabelecimento

Arquitetura de microsserviços para operação de pedidos, pagamento, logística de entrega e rastreamento em tempo real.

Versão	1.0
Data	16 de agosto de 2026
Status	Linha de base aprovada — implementação em Fase 0
Stack	Java 21 · Spring Boot 4.1 · PostgreSQL/PostGIS · MongoDB · RabbitMQ · MQTT · React
Escopo	9 microsserviços de negócio e 1 gateway

Sumário	
1. Sumário executivo	
2. Contexto e escopo	
Problema · Atores · Requisitos · Fora de escopo	
3. Direcionadores arquiteturais	
Atributos de qualidade · Restrições · Premissas	
4. Visão geral da arquitetura	
Estilo · Diagrama de containers · Fluxo de dependências	
5. Decomposição em serviços	
Critério de fronteira · Catálogo · Arquitetura interna	
6. Arquitetura de dados	
Banco por serviço · Persistência poliglota · Modelo do pedido	
7. Comunicação entre serviços	
Síncrona · Assíncrona · Telemetria · Catálogo de eventos	
8. Processos de negócio	
Saga da compra · Máquinas de estado · Compensações · Concorrência	
9. Arquitetura de segurança	
Identidade · Autorização em dois níveis · Proteção de dados	
10. Observabilidade e operação	
11. Estratégia de qualidade	
12. Implantação	
13. Limites conhecidos e evolução	
14. Glossário	

1. Sumário executivo

Este documento descreve a arquitetura de uma plataforma de delivery multiestabelecimento que conecta três públicos com necessidades operacionais distintas: comerciantes que vendem e preparam, entregadores que transportam, e clientes que compram e acompanham.

A solução é composta por **nove microsserviços de negócio e um gateway**, cada um com processo, banco de dados, migrations, testes, contrato e pipeline de implantação próprios. A independência é real e verificável: nenhum serviço acessa o banco de outro, e a integração acontece exclusivamente por API versionada ou evento.

Três características definem esta arquitetura e justificam sua complexidade:

Consistência sem transação distribuída. A compra atravessa quatro serviços — pedido, estoque, pagamento e entrega — sem que exista qualquer transação que os abranja. A integridade é obtida por Saga com compensação explícita, *Transactional Outbox* nos produtores e deduplicação nos consumidores. Cada falha intermediária tem um caminho de reversão definido e testado.

Autorização contextual. Permissão comercial não pertence ao usuário; pertence ao vínculo entre usuário e estabelecimento. A mesma pessoa pode ser administradora de uma loja e colaboradora de outra, com poderes diferentes em cada uma. Isso impede que papéis comerciais residam no token e exige um modelo de autorização em dois níveis.

Separação entre estado operacional e telemetria. O ciclo de vida da entrega e o fluxo de coordenadas GPS têm perfis de carga, durabilidade e criticidade incompatíveis. São tratados por serviços distintos e por canais distintos: eventos de negócio em RabbitMQ com garantia transacional, telemetria em MQTT com protocolo leve e tolerante a rede móvel instável.

PRINCÍPIO DE PROJETO

Complexidade distribuída só se justifica onde há fronteira real de negócio. Onde não há, ela é custo puro. Cada serviço deste desenho existe porque possui vocabulário, ciclo de vida e razão de mudança próprios — não porque a divisão produz um organograma agradável.

2. Contexto e escopo

2.1 Problema

Estabelecimentos precisam vender e operar pedidos sem manter infraestrutura própria. Clientes precisam comparar ofertas de vários estabelecimentos, comprar com segurança e saber onde está o pedido. Entregadores precisam receber ofertas de trabalho com informação suficiente para decidir e executar a entrega com rastreamento confiável.

A plataforma intermedeia os três, assumindo responsabilidade pelo pagamento, pela reserva de estoque, pela atribuição de entregas e pela consistência de tudo isso quando alguma etapa falha.

2.2 Atores

Ator	Responsabilidade no sistema
Cliente	Navega catálogos, monta carrinho, compra e acompanha pedido e entrega
Administrador do estabelecimento	Cria a loja, gerencia equipe e permissões, opera produtos, estoque, pedidos e vendas
Colaborador do estabelecimento	Opera apenas o que o administrador concedeu explicitamente
Entregador	Declara disponibilidade, aceita ofertas, executa e confirma entregas
Administrador da plataforma	Suporte, auditoria, aprovação de entregadores e operação

2.3 Requisitos funcionais

Ref.	Ator	Requisito
R1	Comerciante	Cadastrar-se e criar um estabelecimento
R2	Comerciante	Manter ao menos dois cargos — administrador e colaborador — com permissões distintas
R3	Comerciante	Cadastrar produtos e manipulá-los, incluindo preço, disponibilidade e estoque
R4	Comerciante	Acompanhar vendas, pedidos e o entregador associado
R5	Entregador	Ser notificado de novas entregas
R6	Entregador	Receber destino e valor antes de aceitar
R7	Cliente	Navegar entre produtos de diferentes estabelecimentos
R8	Cliente	Comprar
R9	Cliente	Acompanhar a entrega

2.4 Regra estruturante

Um carrinho e um pedido contêm produtos de **um único estabelecimento**. O cliente navega livremente entre lojas, mas cada compra é fechada com uma delas.

A regra evita que uma única transação produza vários preparos independentes, pagamentos parciais e entregas distintas — o que multiplicaria os estados de falha sem acrescentar valor ao MVP. A evolução natural, quando justificada, é um pedido de marketplace que agrupa subpedidos por estabelecimento, preservando este modelo como unidade.

2.5 Fora de escopo

- Repasse financeiro e conciliação com comerciantes e entregadores
- Cupons, promoções e precificação dinâmica
- Avaliações e reputação
- Agrupamento de entregas e otimização de rotas múltiplas
- Operação em múltiplas cidades ou regiões geográficas

Em todos os casos, os campos de dados necessários existem no modelo desde o início com valor neutro. Acrescentar coluna depois é barato; alterar o significado de um valor já cobrado e já contabilizado, não.

3. Direcionadores arquiteturais

3.1 Atributos de qualidade

Ordenados por prioridade. Onde há conflito entre dois atributos, o de maior prioridade prevalece — e essa ordem é o que sustenta boa parte das decisões descritas adiante.

#	Atributo	Como se manifesta e como é atingido
1	Integridade financeira	Nenhuma cobrança sem contrapartida, nenhuma entrega sem pagamento confirmado, nenhum efeito duplicado por reprocessamento. Obtido com Saga compensada, Outbox transacional, idempotência no consumidor e valor cobrado derivado exclusivamente do pedido.
2	Segurança e privacidade	Isolamento entre estabelecimentos, proteção contra acesso indevido por identificador, e tratamento de endereço e localização como dado pessoal sensível. Obtido com autorização em dois níveis e verificação de posse no serviço dono do recurso.
3	Consistência eventual observável	Divergência temporária é aceitável; divergência permanente e silenciosa não é. Todo estado intermediário tem prazo, e toda falha tem compensação e métrica.
4	Evolutividade	Trocar provedor de mapas, de pagamento ou de emissão de token não deve alcançar a regra de negócio. Obtido com arquitetura hexagonal e integrações atrás de portas.
5	Disponibilidade	Falha de um serviço degrada uma capacidade, não a plataforma. Obtido com comunicação assíncrona onde possível, e timeout e disjuntor onde a chamada precisa ser síncrona.
6	Desempenho	Navegação de catálogo é a operação de maior volume e é otimizada com cache e paginação. Telemetria de alta frequência não compartilha caminho com transação de negócio.

3.2 Restrições

- **Ambiente único de desenvolvimento.** Toda a plataforma precisa ser executável em uma estação de trabalho, o que impõe perfis de inicialização parcial e uma instância de banco com esquemas logicamente separados.
- **Equipe reduzida.** A arquitetura precisa ser construível em incrementos verticais, cada um entregando capacidade completa e testável.
- **Rastreamento em navegador.** O aplicativo do entregador roda como aplicação web, o que limita a coleta de GPS em segundo plano e obriga o desenho a tolerar lacunas de telemetria.

3.3 Premissas

- Volume compatível com operação regional — dezenas de estabelecimentos e milhares de pedidos por dia, não milhões.
- O provedor de pagamento confirma transações por *webhook* assinado, de forma assíncrona.
- Provedores de geocodificação e roteamento são substituíveis e podem ficar temporariamente indisponíveis sem impedir a conclusão de uma entrega.

4. Visão geral da arquitetura

4.1 Estilo arquitetural

Microserviços orientados a domínio, com comunicação híbrida: síncrona para consulta e comando de efeito imediato, assíncrona para propagação de fatos consumados, e um canal dedicado de telemetria para dados de alta frequência.

Internamente, cada serviço adota arquitetura hexagonal, com o domínio isolado de framework, ORM e protocolo.

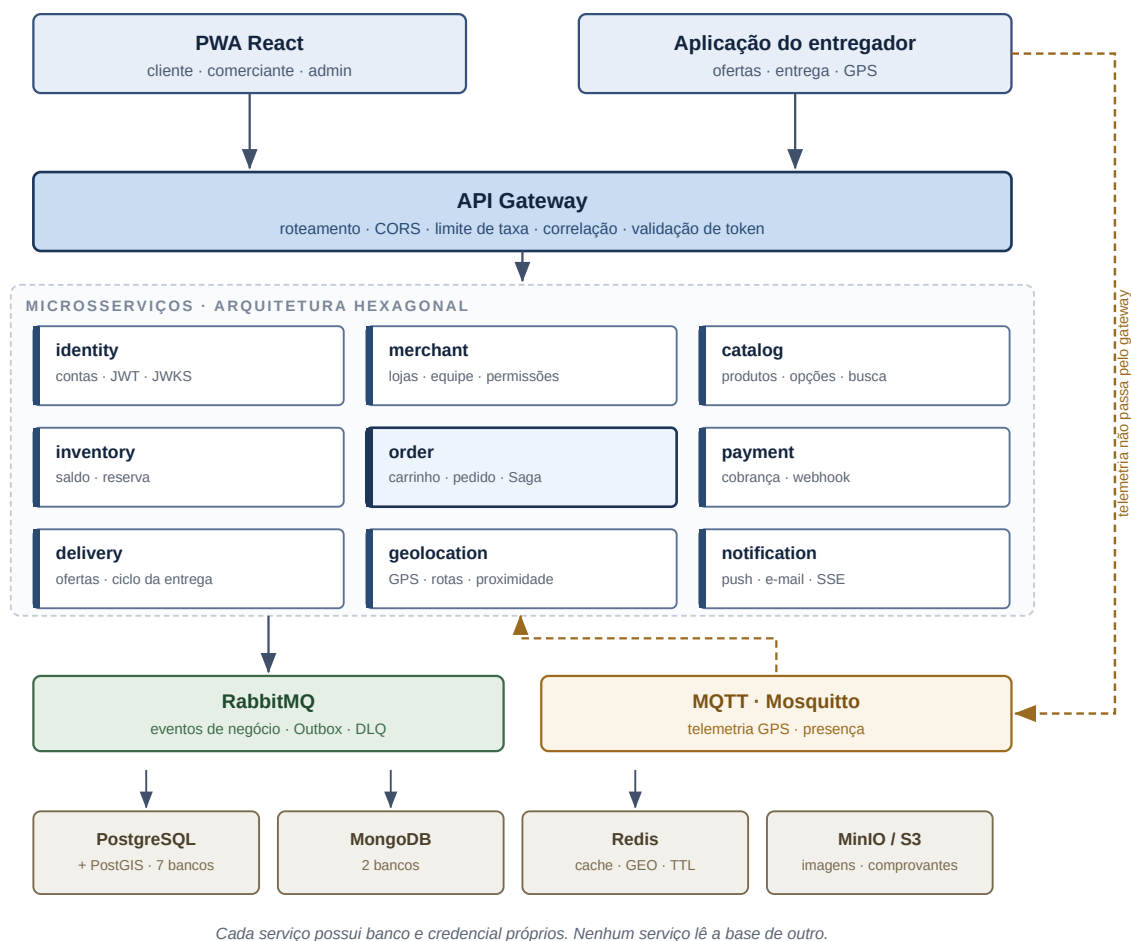


Figura 1. Visão de containers. Toda operação HTTP externa atravessa o gateway; a telemetria GPS do entregador segue por MQTT, deliberadamente fora do caminho transacional. O **order-service** aparece destacado por ser o orquestrador da Saga da compra.

4.2 Regra de dependência entre camadas

Dentro de cada serviço, a divisão de pacotes é por anel de dependência, não por camada técnica. As setas apontam para dentro: a infraestrutura conhece o domínio, nunca o contrário.

```
api ————— v
                |
infrastructure —> application —> domain
```

- **domain** — regra de negócio pura. Não importa Spring, JPA, MongoDB nem classe de outro serviço.
- **application** — casos de uso e orquestração; depende de interfaces declaradas como portas.
- **infrastructure** — persistência, mensageria, clientes HTTP e segurança; implementa as portas.
- **api** — traduz HTTP em comando e resultado em resposta.

Entidades de persistência residem em `infrastructure/persistence/entity`, separadas do modelo de domínio. A duplicação é intencional: ela impede que decisões de mapeamento relacional contaminem a regra de negócio.

VERIFICAÇÃO AUTOMÁTICA

Cada serviço contém um teste de arquitetura que falha o build se a regra de dependência for violada — por exemplo, se uma classe de domínio importar Spring ou receber anotação de persistência. Arquitetura mantida apenas por revisão manual erode em poucas semanas; mantida por teste, não.

5. Decomposição em serviços

5.1 Critério de fronteira

Um serviço existe quando satisfaz simultaneamente três condições: possui vocabulário próprio, muda por razões próprias, e é dono de um conjunto de dados que ninguém mais escreve.

O caso mais ilustrativo é a separação entre entrega e geolocalização. Ambas tratam de "onde está o entregador", mas por motivos diferentes: o **delivery-service** governa o *estado* da entrega — oferta, aceite, retirada, conclusão — enquanto o **geolocation-service** governa *coordenadas e cálculo espacial*. Uni-los produziria um serviço que mistura regra operacional, integração cartográfica e ingestão de alta frequência, com três perfis de carga incompatíveis no mesmo processo.

5.2 Catálogo de serviços

Serviço	Responsabilidade	Dados próprios
identity-service	Contas, credenciais, emissão de token assimétrico, publicação de chave pública, papéis globais, bloqueio e revogação	usuários, papéis, tokens de renovação, tentativas de acesso
merchant-service	Estabelecimentos, equipe, convites, permissões por estabelecimento, horário de funcionamento e endereço de retirada	estabelecimentos, vínculos, permissões, convites
catalog-service	Categorias, produtos, grupos de opções, preço, disponibilidade, busca e cotação de itens	produtos, categorias, imagens
inventory-service	Saldo, reserva temporária, expiração, confirmação de baixa e movimentações	saldos, reservas, movimentações
order-service	Carrinho, pedido, itens congelados, totais e orquestração da Saga da compra	pedidos, itens, opções do item, carrinhos
payment-service	Cobrança, integração com adquirente, webhook assinado, estorno e máquina de estados do pagamento	pagamentos, tentativas, recibos de webhook, estornos
delivery-service	Perfil e disponibilidade do entregador, ofertas, aceite atômico e ciclo operacional da entrega	entregadores, entregas, ofertas, histórico de status
geolocation-service	Ingestão e validação de telemetria, geocodificação, proximidade, rota, distância, ETA e transmissão de rastreamento	amostras de localização, áreas de atendimento, endereços geocodificados, cache de rota
notification-service	Push, e-mail e canal em tempo real; preferências, modelos e histórico de tentativas	dispositivos, preferências, modelos, tentativas

O **gateway** não contém regra de negócio. Concentra roteamento, política de origem cruzada, limite de taxa, injeção de identificador de correlação e a primeira validação do token.

5.3 Acoplamento deliberado

O **merchant-service** é consultado por catálogo, estoque, pedido e entrega sempre que uma operação exige permissão comercial. Trata-se do ponto de maior acoplamento da plataforma, assumido conscientemente por três razões: a informação é pequena e altamente cacheável; ela muda com frequência incompatível com sua permanência no token; e a alternativa — replicar o modelo de permissões em cinco serviços — produziria divergência de autorização, que é a pior classe de defeito de segurança.

A mitigação é explícita: decisão de autorização cacheada em Redis com validade curta, invalidação ativa ao consumir o evento de alteração de vínculo, e política de negação em caso de indisponibilidade. A consequência é aceita: com o **merchant-service** fora do ar, a plataforma não aceita escritas comerciais.

6. Arquitetura de dados

6.1 Banco por serviço

Cada serviço possui banco e credencial próprios. Um serviço nunca consulta a base de outro; a integração acontece por API ou evento. Em desenvolvimento, uma instância de PostgreSQL hospeda sete bancos logicamente separados e uma instância de MongoDB hospeda dois — a separação lógica é a mesma; muda apenas a topologia física.

A consequência prática mais relevante é que consultas que atravessam fronteiras deixam de existir. "Pedidos que contêm produtos do afiliado X" não é uma junção; é ou uma chamada ao serviço dono, ou um dado denormalizado no momento da compra. Este desenho escolhe a segunda opção sempre que o dado pertence ao histórico.

6.2 Persistência poliglota

Tecnologia	Onde	Por quê
PostgreSQL	identity, merchant, inventory, order, payment, delivery	Transação ACID, integridade referencial, restrição de unicidade e controle de concorrência — requisitos de todo dado financeiro ou de saldo
PostGIS	geolocation	Polígonos de área de atendimento, geofences e consulta espacial indexada. Distância geográfica nunca é calculada tratando coordenadas como plano cartesiano
MongoDB	catalog, notification	Estrutura variável por categoria de produto — tamanhos, adicionais, combos — e modelos de notificação por canal
Redis	transversal	Carrinhos com expiração, última posição e proximidade, chaves de idempotência, limite de taxa e cache de decisão de permissão
Objetos (S3)	catalog, delivery	Imagens e comprovantes, com URL assinada e validade — binário não ocupa banco transacional

Redis não é fonte de verdade de pedido, pagamento ou saldo. Toda chave possui prazo de expiração, e travas distribuídas nele são defesa complementar: a garantia final permanece nas restrições e transações do banco relacional.

6.3 Versionamento de esquema

Nenhuma estrutura é criada pela aplicação em tempo de execução. Serviços relacionais operam com validação de esquema ativa e migrations versionadas; serviços documentais versionam índices, validação e políticas de expiração com a mesma disciplina. Um esquema que existe sem estar versionado é um esquema que ninguém consegue reproduzir.

6.4 Modelo do pedido

O pedido é o agregado central da plataforma e concentra as decisões de modelagem mais consequentes.

Congelamento do item

O carrinho armazena apenas identificadores e quantidades. No fechamento, o catálogo valida a seleção contra as regras de cada grupo de opções e devolve as linhas precificadas; o pedido então **congela** nome do produto, preço-base, e —

para cada opção escolhida — nome do grupo, nome da opção e o acréscimo correspondente.

Congelar os nomes, e não apenas os valores, é o que permite reemitir um pedido antigo com exatamente o texto que o cliente viu, mesmo após renomeações no catálogo. E como o preço é sempre resolvido no servidor a partir do catálogo, nenhum valor enviado pelo cliente influencia a cobrança.

Composição do valor

subtotal dos itens	
+ taxa de entrega	cobrada do cliente
+ taxa de serviço	
+ gorjeta	
- desconto	
= total	único valor que o serviço de pagamento aceita cobrar

Três invariantes são verificadas na construção do agregado, não em teste: o total é a soma dos componentes já arredondados; o subtotal corresponde à soma dos itens; e nenhum componente é negativo. Arredondar apenas ao formar cada componente — e somar valores já arredondados — é o que evita a divergência clássica entre o valor exibido e o valor cobrado.

DISTINÇÃO ESSENCIAL

A taxa de entrega cobrada do cliente e o valor pago ao entregador são **campos distintos**. A diferença entre eles é margem ou subsídio da plataforma. Tratá-los como um único valor impossibilita frete grátis, entrega subsidiada e qualquer política de preço — e é irreversível depois que existirem pedidos gravados.

Referência geográfica congelada

O pedido guarda o endereço textual e as coordenadas de retirada e de destino como valores imutáveis, sem tipo espacial e sem consulta posterior ao serviço de geolocalização. Se o cliente editar o endereço cadastrado após a compra, a rota do pedido antigo permanece a que foi acordada.

7. Comunicação entre serviços

7.1 Síncrona

Usada quando o chamador precisa da resposta para prosseguir: consulta de permissão comercial, cotação de itens no catálogo e cálculo de rota.

Toda chamada carrega prazo máximo de espera, disjuntor e propagação de contexto de rastreamento. O comportamento padrão de bibliotecas HTTP — esperar indefinidamente — transforma a lentidão de um serviço em indisponibilidade de todos os que dependem dele; por isso o prazo é obrigatório, não opcional.

Nenhum modelo Java é compartilhado entre serviços. Quando conveniente, clientes são gerados a partir do contrato publicado.

7.2 Assíncrona — eventos de negócio

Usada para propagar fatos consumados: estoque reservado, pagamento aprovado, pedido pronto, entrega concluída. O canal é durável, com confirmação de publicação, reconhecimento manual, política de nova tentativa e fila de mensagens mortas.

Publicação transacional

O produtor grava o dado de domínio e o evento na **mesma transação**, em uma tabela de saída. Um processo separado publica no barramento a partir dessa tabela. Sem isso, existe uma janela em que o pedido foi gravado e o evento se perdeu — inconsistência que nenhum mecanismo posterior reconcilia automaticamente.

Consumo idempotente

O barramento entrega ao menos uma vez. Todo consumidor registra o identificador da mensagem antes de aplicar efeito irreversível, de modo que reprocessamento não duplique cobrança, baixa de estoque ou entrega.

A regra decorrente é mecânica e vale sem exceção: **quem publica precisa de tabela de saída; quem consome precisa de registro de mensagens processadas.**

7.3 Telemetria

Coordenadas do entregador seguem por MQTT, não pelo barramento de negócio. A razão é dupla: o volume de telemetria é ordens de grandeza maior que o de eventos transacionais e os misturaria; e o protocolo é projetado para rede móvel instável, com sessão persistente, níveis de qualidade de serviço e mensagem de última vontade para detectar desconexão.

Cada dispositivo autentica-se individualmente e só pode publicar no próprio tópico — restrição verificada no broker e novamente no serviço, comparando o identificador do tópico com a identidade autenticada. Coordenadas são validadas quanto a horário, sequência, precisão e plausibilidade de deslocamento antes de serem aceitas.

7.4 Tempo real para o cliente

O navegador nunca recebe credencial de broker interno. O acompanhamento de status e posição chega por fluxo autenticado no gateway, alimentado pelo serviço de geolocalização.

7.5 Catálogo de eventos

Evento	Produtor	Consumidores
Usuário registrado	identity	notification
Estabelecimento criado	merchant	catalog, notification
Vínculo de equipe alterado	merchant	serviços autorizadores
Produto alterado	catalog	order, cache de busca
Estoque reservado / rejeitado / liberado	inventory	order
Pagamento aprovado / recusado	payment	order, notification
Pedido pago	order	merchant, notification
Pedido pronto para retirada	order	delivery
Oferta de entrega criada	delivery	notification
Entrega atribuída / status alterado / concluída	delivery	order, notification
Entregador entrou em geofence	geolocation	delivery, notification

Todo evento carrega identificador único, tipo, versão, instante de ocorrência, identificador de correlação e conteúdo. Mudança incompatível cria nova versão; a anterior permanece publicada enquanto houver consumidor dela. Contrato de evento é mais difícil de alterar que contrato REST, porque o consumidor pode estar processando uma mensagem antiga ainda na fila.

8. Processos de negócio

8.1 Saga da compra

A compra atravessa quatro serviços sem transação abrangente. A sequência é orquestrada pelo **order-service**, que mantém o estado e decide as compensações.

#	Etapa	Descrição
1	Cotação e congelamento	O catálogo valida a seleção de opções e devolve linhas precificadas; o pedido congela itens, opções, valores e pontos geográficos
2	Reserva de estoque	Somente os itens com controle de estoque são reservados, de forma indivisível sobre o conjunto
3	Cobrança	O pagamento é solicitado pelo valor total do pedido; a confirmação chega por webhook assinado
4	Confirmação da baixa	Aprovado o pagamento, a reserva é comprometida antes de o pedido ser declarado pago
5	Preparo	O estabelecimento recebe o pedido, prepara e sinaliza que está pronto
6	Atribuição da entrega	Candidatos próximos são consultados, ofertas emitidas e um entregador vence o aceite
7	Execução e conclusão	Retirada, trânsito com telemetria e confirmação de entrega

Caminho sem controle de estoque

Grande parte do que se vende em delivery — notadamente comida preparada — não possui saldo contado. Pedidos cujos itens são todos dessa natureza **não passam pela etapa de reserva**: nascem aguardando pagamento, sem qualquer chamada ao serviço de estoque. Pedidos mistos reservam apenas a parcela controlada. Tratar o caso mais comum como exceção seria pagar custo de rede na maioria das compras e manter o sistema em um estado que não descreve o que está acontecendo.

Janela de reserva

A reserva expira em prazo deliberadamente maior que a janela de pagamento, com folga para absorver latência de webhook e novas tentativas. Ao receber a aprovação, o pedido compromete a reserva antes de se declarar pago; se o comprometimento falhar por expiração, dispara-se estorno imediato e o cancelamento registra a causa. Com a folga adotada isso não deveria ocorrer — e é justamente por isso que a ocorrência é medida e alertada.

8.2 Ciclo de vida do pedido

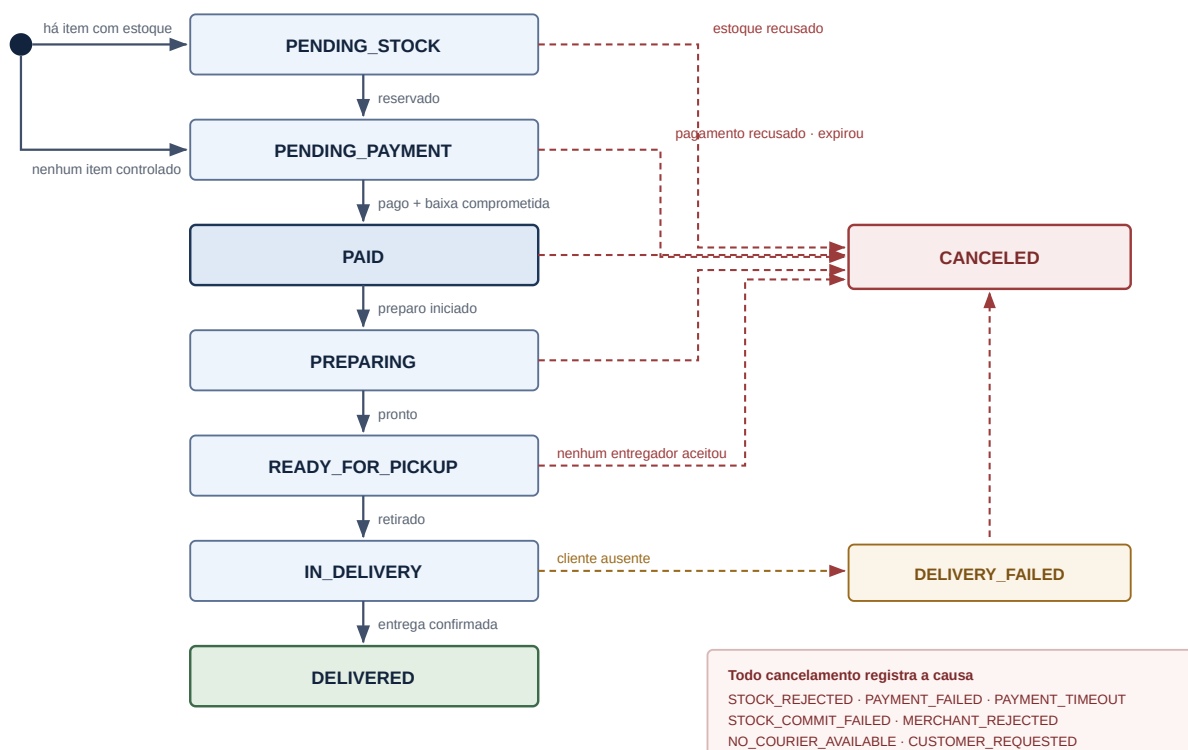


Figura 2. Ciclo de vida do pedido. Duas entradas distintas conforme haja ou não item com controle de estoque. Todo estado anterior à entrega possui saída para cancelamento — inclusive **READY_FOR_PICKUP**, cuja ausência deixaria um pedido pago sem transição possível quando nenhum entregador aceitasse a oferta. Cancelamentos posteriores à cobrança implicam estorno.

8.3 Compensações

Falha	Compensação	Estorno
Estoque recusado	Cancelar antes de qualquer cobrança	não
Pagamento recusado	Liberar reserva e cancelar	não
Pagamento não confirmado no prazo	Liberar reserva e cancelar por expiração	não
Reserva expirada após aprovação	Estornar imediatamente, cancelar e alertar	sim
Estabelecimento cancela após pagamento	Liberar ou ajustar estoque e estornar	sim
Cliente cancela na janela permitida	Liberar reserva e estornar	sim
Nenhum entregador aceita	Ampliar raio, repetir rodada e, esgotadas as tentativas, cancelar e notificar operação	sim
Entrega não concluída	Nova tentativa ou encerramento conforme política	conforme

Mensagem duplicada	Descartar por identificador já processado	não
--------------------	---	-----

8.4 Concorrência

Dois pontos do sistema sofrem disputa real e recebem tratamento explícito.

Reserva de estoque. Ler o saldo e depois gravar o decremento permite que duas requisições simultâneas ultrapassem a verificação e vendam o mesmo item. A operação é atômica, com trava ou atualização condicional, e a restrição de não-negatividade permanece no banco como última linha de defesa.

Aceite de oferta. Dois entregadores podem aceitar a mesma entrega no mesmo instante; apenas um deve vencer. O aceite é uma atualização condicional sobre o estado da oferta, e o perdedor recebe resposta de conflito, não erro.

9. Arquitetura de segurança

9.1 Identidade

O **identity-service** é o único emissor de token da plataforma. Assina com chave assimétrica e publica a chave pública em endpoint padronizado; os demais serviços validam localmente, sem consultá-lo a cada requisição.

A escolha por assinatura assimétrica é deliberada. Com segredo compartilhado entre nove serviços, qualquer um deles — ou qualquer vazamento em qualquer um deles — permitiria forjar token de qualquer usuário. Com par de chaves, apenas o emissor pode assinar.

O token carrega identificador do usuário, papéis globais, emissor, destinatário, validade e identificador único. Não carrega senha, endereço, lista de estabelecimentos nem permissões comerciais — informação que muda com frequência não pertence a um token de vida útil fixa.

O acesso é de curta duração e renovado por token de renovação rotativo. A detecção de reúso de um token já consumido revoga a família inteira, tratando o evento como indício de comprometimento.

9.2 Autorização em dois níveis

Nível	Fonte	Responde
Global	Papel no token	Este usuário é cliente, entregador ou administrador da plataforma?
Contextual	Vínculo no merchant-service	Este usuário pode executar esta ação <i>neste</i> estabelecimento?

A separação é imposta pelo domínio: a mesma pessoa pode administrar uma loja e colaborar em outra, com poderes distintos. Permissão comercial é atributo do vínculo, não do usuário.

Regras de contenção

- Nenhum serviço confia em identificador recebido pela URL sem confrontá-lo com o usuário autenticado e as permissões aplicáveis.
- Quem gerencia equipe pode administrar colaboradores, nunca administradores, e não pode conceder permissão que não possui.
- Um estabelecimento sempre mantém ao menos um administrador ativo.
- Consultas são invariavelmente limitadas pelo estabelecimento ou pelo proprietário autenticado, e não por parâmetro do cliente.

9.3 Proteção de dados

- **Valores.** Preço e total são sempre recalculados no servidor; valor recebido do cliente é ignorado.
- **Endereço.** Dados completos do cliente são liberados apenas ao entregador efetivamente atribuído, e somente após o aceite. Antes disso, a oferta apresenta região, distância e valor.

- **Localização.** Rastreamento apenas de entregadores em entrega ativa, com política de retenção, redução de precisão em visualização pública e anonimização em uso analítico.
- **Registros.** Token, senha, documento, dado de pagamento e coordenada exata não entram em log.
- **Segredos.** Nenhuma credencial no repositório; variáveis de ambiente em desenvolvimento e cofre gerenciado em produção, com rotação periódica e varredura automatizada no pipeline.
- **Webhook.** Assinatura validada e processamento idempotente — o endereço é público e a mensagem pode ser repetida.

9.4 Exclusão e retenção

Pedido de exclusão de conta conflita com a obrigação de reter histórico de venda. O desenho resolve por anonimização: os dados pessoais do cliente são removidos ou substituídos, e o pedido permanece íntegro para fins contábeis, sem vínculo identificável.

10. Observabilidade e operação

Com nove serviços e dois canais de mensageria, a pergunta operacional deixa de ser "o serviço está no ar" e passa a ser "onde este pedido parou". Responder isso exige correlação, e correlação exige preparo desde o início.

10.1 Correlação

Toda requisição e toda mensagem carregam identificador de rastreamento, de sequência e de correlação de negócio. O identificador de correlação é injetado no gateway e propagado inclusive dentro do conteúdo dos eventos — retrofitar isso em um catálogo de eventos já publicado é caro, e por isso é tratado como requisito de código, não como configuração de infraestrutura.

10.2 Sinais

Sinal	Finalidade	Exemplos relevantes
Métricas	Tendência e alerta	Pedidos por estado; taxa de aprovação de pagamento; tempo até encontrar entregador; ofertas recusadas e expiradas; profundidade da fila de mensagens mortas; atraso na publicação da tabela de saída
Rastros	Diagnóstico de caminho	Percurso completo de uma compra atravessando pedido, estoque, pagamento e entrega
Registros	Contexto de exceção	Estruturados, com ambiente, serviço e contexto de rastreamento; sem dado sensível
Saúde	Orquestração	Sondas de vivacidade e de prontidão distintas — prontidão considera dependências essenciais

10.3 Indicadores de anomalia

Alguns valores não indicam apenas desempenho, mas defeito de consistência, e merecem alerta imediato:

- Reserva expirada após aprovação de pagamento — indica janela mal dimensionada e produz estorno evitável.
- Crescimento da fila de mensagens mortas — indica contrato divergente ou consumidor com defeito.
- Atraso na publicação da tabela de saída — indica que fatos ocorreram e não foram propagados.
- Idade da última coordenada em entrega ativa — indica perda de telemetria, não necessariamente de conexão.

11. Estratégia de qualidade

Nível	Alcance	Verifica
Domínio	Sem framework, sem banco	Cálculo de totais e invariantes; transições de estado válidas e inválidas; regras de permissão; expiração de reserva; validação de coordenada. É a maior parte da suíte e executa em milissegundos
Arquitetura	Todo módulo	Regra de dependência entre camadas; ausência de framework no domínio; ausência de anotação de persistência em classe de domínio
Integração	Tecnologias reais em contêiner	Migrations; consulta espacial; concorrência de reserva; aceite simultâneo de oferta; publicação transacional; idempotência de consumo
Contrato	Produtor e consumidor	Compatibilidade de API e de esquema de evento; mudança incompatível falha o pipeline
Ponta a ponta	Jornada completa	Compra com sucesso; falha de estoque antes da cobrança; recusa de pagamento liberando reserva; disputa de aceite; publicação de telemetria com restrição de tópico

Testes de banco executam contra a mesma tecnologia usada em produção. Banco em memória, por acelerar a suíte, produziria aprovação em cenários que o banco real recusaria — precisamente nos pontos de concorrência e consulta espacial que mais importam neste sistema.

11.1 Cenários obrigatórios

1. Colaborador sem permissão específica recebe negativa de autorização
2. Produto de um estabelecimento não é manipulável por outro
3. Carrinho recusa item de segundo estabelecimento
4. Seleção de opções que viola mínimo ou máximo é recusada
5. Preço alterado entre carrinho e fechamento exige reconfirmação
6. Estoque insuficiente encerra a Saga antes de qualquer cobrança
7. Pedido sem item controlado não aciona o serviço de estoque
8. Recusa de pagamento libera a reserva
9. Evento entregue duas vezes produz efeito único
10. Dois entregadores aceitam a mesma oferta e apenas um vence
11. Entregador não consegue publicar telemetria no tópico de outro
12. Coordenada duplicada ou fora de ordem não retrocede o rastreamento

12. Implantação

Cada serviço é empacotado como imagem de contêiner construída em múltiplos estágios, executada com usuário sem privilégio e com verificação de saúde declarada. A imagem é identificada pelo commit que a originou, analisada quanto a vulnerabilidades e acompanhada de inventário de componentes.

Cada serviço possui pipeline próprio, acionado por alteração em seu próprio caminho. O pipeline compila, executa os cinco níveis de teste, valida contratos, analisa dependências e segredos, constrói e analisa a imagem, e só publica se todas as verificações passarem.

12.1 Ambiente de desenvolvimento

A plataforma completa envolve dezenas de contêineres, o que a torna pesada para uso cotidiano. A composição é dividida em perfis, de modo que o trabalho diário exija apenas a infraestrutura mais o serviço em desenvolvimento — os demais permanecem desligados.

O ambiente de desenvolvimento usa as mesmas tecnologias de produção. A tentação de substituir o banco relacional por um equivalente em memória é recusada deliberadamente: ela criaria uma garantia falsa exatamente onde o sistema é mais sensível.

12.2 Evolução da topologia

Orquestração de contêineres, escalonamento automático e malha de serviços não são requisitos para demonstrar que a plataforma é composta por microsserviços. O critério arquitetural é a independência de processo, dados, contrato e implantação — que já está satisfeita. Orquestração entra quando houver necessidade real de escala e operação contínua.

13. Limites conhecidos e evolução

Documentar o que a arquitetura ainda não resolve é parte de descrevê-la. Os pontos abaixo são conhecidos, aceitos para o escopo atual, e têm caminho de evolução definido.

Limite	Consequência	Evolução
Dependência do serviço de estabelecimentos	Sua indisponibilidade impede escritas comerciais em toda a plataforma	Projeções locais de autorização alimentadas por evento, substituindo a consulta síncrona
Cotação de entrega por distância geodésica	A taxa inicial subestima o percurso viário e é sistematicamente otimista	Substituição do provedor de cotação por cálculo de rota real, sem alteração no domínio
Um estabelecimento por pedido	Compra em duas lojas exige dois pedidos	Pedido de marketplace agregando subpedidos, preservando o modelo atual como unidade
Rastreamento em navegador	Coleta de GPS é interrompida com o aplicativo em segundo plano	Aplicativo nativo ou híbrido para o entregador, se o rastreamento contínuo se tornar requisito rígido
Repasse financeiro ausente	A plataforma cobra, mas não liquida com comerciante e entregador	Serviço de carteira e conciliação, com razão contábil próprio
Busca de catálogo por índice do banco	Adequada ao volume previsto; limitada em relevância e tolerância a erro de digitação	Motor de busca dedicado, alimentado por evento de alteração de produto

13.1 Ordem de construção

A arquitetura é construída em incrementos verticais, cada um entregando capacidade completa e testável, e não por camadas horizontais.

Incremento	Entrega	Requisitos atendidos
1–2	Contas, autenticação assimétrica e publicação de chave	R1
3	Estabelecimentos, equipe, permissões e gateway	R1, R2
4	Interface web mínima	—
5	Catálogo, opções e navegação pública	R3, R7
6	Estoque, carrinho, pedido, Saga e publicação transacional	R4 parcial, R8 parcial
7	Pagamento, webhook e compensações completas	R8
8	Entregadores, ofertas, ciclo da entrega e acompanhamento por estado	R4, R5, R6, R9

9	Telemetria, geoprocessamento, rota e rastreamento em mapa	R9 ampliado
10–12	Notificação multicanal, observabilidade e endurecimento	R5 ampliado

Os nove requisitos funcionais estão integralmente atendidos ao término do oitavo incremento. O nono — geoprocessamento e rastreamento em mapa — refina a experiência de acompanhamento, que até então é servida por transição de estado. Essa distinção é registrada deliberadamente: ela permite decidir, com informação, até onde levar o sistema.

14. Glossário

Termo	Definição no contexto deste sistema
Saga	Sequência de operações locais em serviços distintos, coordenada por um orquestrador, na qual cada passo possui uma ação compensatória para o caso de falha posterior
Compensação	Operação que desfaz o efeito de um passo já concluído, quando um passo seguinte falha. Não é rollback: é uma nova operação de negócio
Tabela de saída	Tabela na qual o evento é gravado na mesma transação do dado de domínio, garantindo que fato e notificação não divirjam
Idempotência	Propriedade pela qual processar a mesma mensagem mais de uma vez produz o mesmo resultado que processá-la uma vez
Congelamento	Cópia imutável, no momento da compra, de nome, preço e opções do item, tornando o pedido independente de alterações futuras no catálogo
Reserva	Retenção temporária de saldo, com prazo de expiração, entre o fechamento do pedido e a confirmação do pagamento
Autorização contextual	Permissão derivada do vínculo entre usuário e estabelecimento, e não de um papel global do usuário
Porta e adaptador	Interface declarada pelo domínio e implementada pela infraestrutura, permitindo substituir provedor externo sem alterar regra de negócio
Geocodificação	Conversão de endereço textual em coordenadas geográficas
Geofence	Área virtual cuja entrada ou saída dispara um evento — utilizada para detectar chegada ao estabelecimento
Fila de mensagens mortas	Destino de mensagens que falharam além do número máximo de tentativas, preservadas para análise

SOBRE A MANUTENÇÃO DESTE DOCUMENTO

Este documento descreve a arquitetura em vigor. As decisões que a produziram — com contexto, alternativas avaliadas e consequências aceitas — são mantidas separadamente, em registros versionados junto ao código. Quando uma decisão for revista, este documento reflete o novo estado; o registro preserva o histórico do raciocínio.